第 3 讲：循环、跳转表与过程调用

ICS 第一次小测复习 · 发布版 · 2026-10-13 · 独立答案另册

1. 读控制流，先找“下一步去哪里”
2. switch：先找入口，再沿代码走
3. 调用：把“去哪里”和“带什么”分开
4. ABI：函数之间约定怎么交接
5. 递归：重复的是代码，分开保存的是每次调用的状态
6. 遇到长题时，按这个顺序落笔
7. 独立练习：4 道必做，共 40 分
8. 本讲完成时应留下什么

来源与范围核对

本讲对应复习的第 3 天，主要覆盖课程第 5 讲的循环与 switch，以及课程第 6 讲的过程调用。今天要建立两种能力：看到一段反复跳转的代码，能还原“什么在重复、何时停止”；看到函数调用，能画出“控制去了哪里、什么值必须留下、返回后从哪里继续”。不用背长汇编，必须能追踪状态。

先读循环与跳转表，再读调用与递归。每部分的示范例题有完整推理；最后的四道必做题请独立作答后再看练习答案。必做总分 40 分，32 分为本日基础过关线，另有一道选做题。今天的任务不含 2025 年第一次小测原题。

统一前提。 以下采用 x86-64、Linux、System V AMD64 ABI 和 AT&T 汇编语法，`long` 与指针均为 8 字节。除专门讨论无符号索引外，C 中的 `long` 为有符号整数。示例给出有限输入范围，避免有符号溢出、除零等未定义行为。汇编是为讲解整理的等价实现，不声称某个编译器版本一定逐字生成它。不能在 Apple Silicon 或 Windows 的本机汇编上直接套用本讲的调用规则。

1. 读控制流，先找“下一步去哪里”

普通指令执行完，控制流通常走到下一条指令。`jmp .Lx` 改成无条件去标签 `.Lx`；`jl .Lx` 则只有条件成立时才去那里，否则继续向下。标签是代码位置的名字，不是变量。标签之间没有“隔墙”：如果前面没有跳走或返回，就会继续执行下一个标签后的代码。

读汇编时先复习三个最容易混淆的写法：`movq %rsi,%rax` 把值从 `%rsi` 复制到 `%rax`；`addq %rdx,%rax` 把 `%rax` 改成原来的 `%rax+%rdx`；`cmpq %rdi,%rdx` 按 `%rdx-%rdi` 的结果设置条件码，却不修改两个操作数。因此，后接 `jl` 表示按有符号数比较 `%rdx<%rdi`，不是反过来。

一段循环最值得先标出来的是四件事：**初始化、循环条件、循环体、更新**。初始化只在进入循环前执行一次；循环体是希望重复的工作；更新让状态向结束条件推进；条件决定下一轮是否执行。代码中这四件事可能被拆到不同位置，不能因为条件写在汇编末尾就认定原来是 `do-while`。

1.1 三种 C 循环，区别在第一次检查发生在哪里

`while` 先检查条件，再决定是否执行循环体。因此，它可能一次也不执行：

```
while (x > 0) {
    work(x);
    x--;
}
```

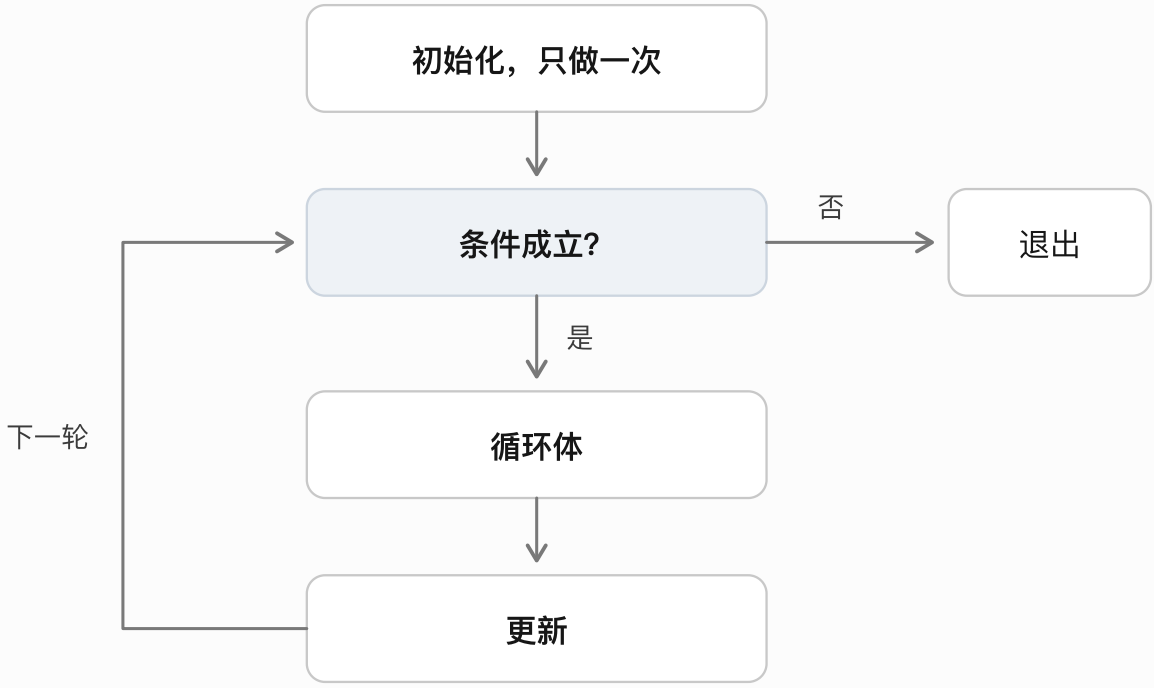
这里若开始时 `x==0`，就直接结束。`do-while` 先执行一次循环体，再检查条件：

```
do {
    work(x);
    x--;
} while (x > 0);
```

无论起始的 x 是什么，第一次 work(x) 都会发生。考试中，零值输入经常就是区分两种结构的最小反例。

for（初始化；条件；更新）循环体 的执行顺序是：初始化；检查条件；循环体；更新；再次检查条件。省略条件相当于条件始终为真。先把下面的图读懂，再去看汇编中这些部分摆在哪里。

循环执行的顺序



沿箭头走一轮：初始化没有回边；循环体执行完后，先更新，再回到条件。

读图时沿箭头走：初始化没有回来的箭头，所以只执行一次；“更新→条件”的返回箭头形成循环。若循环体执行 break，应直接去“退出”；若 for 的循环体执行 continue，应先去“更新”，再检查条件。直接把 for 改成 while 而忘了 continue 的去向，会改变程序行为。

1.2 条件在汇编末尾，也可能是 while

编译器可以先跳到末尾检查条件，满足后再回到前面的循环体。这叫“先跳到测试位置”的布局：

```
初始化
jmp .Ltest          # 第一次先判断，不先执行循环体
.Lbody:
    循环体
    更新
.Ltest:
    比较
    条件成立则跳回 .Lbody
结束
```

另一个常见布局是先在入口检查一次，不满足就结束；接下来循环体放在前面，循环条件放在后面。两种布局都能实现 while。所以还原 C 时，不必猜编译器原来用了哪个关键字；先写出行为等价、边界正确的循环。

所谓“回边”，就是从后面的代码跳回前面的代码。它能帮助定位循环，但不是完整答案。必须继续问：第一次如何到达这里？哪个寄存器变化？比较发生在更新前还是后？等号算不算继续？这些问题决定会不会少一轮、多一轮，或在输入零时错误地进入循环。

示例 D3-E01：从汇编恢复一个求和循环（助手补充）

输入 `0 <= n <= 1000`，`n` 在 `%rdi`，返回 `long`。给定：

```
sum_before_n:
    movl $0, %eax
    movl $0, %edx
    jmp .Lsum_test
.Lsum_body:
    addq %rdx, %rax
    addq $1, %rdx
.Lsum_test:
    cmpq %rdi, %rdx
    jl .Lsum_body
    ret
```

第一步，给寄存器起临时变量名。 `%rax` 初始为零，在每一轮累加 `%rdx`，最后又是返回寄存器，很像累计值 `sum`。`%rdx` 初始为零，每轮加一，很像计数器 `i`。给它们起名字只是帮助理解，最终还要靠整段代码验证。

第二步，翻译比较。 `cmpq %rdi,%rdx` 比较 `i-n`；`jl` 在 `i<n` 时进入循环体。这是有符号比较，与题目中的 `long` 一致。最前面的 `jmp .Lsum_test` 说明先判断，再执行循环体。

第三步，拼回 C。

```
long sum_before_n(long n) {
    long sum = 0;
    for (long i = 0; i < n; i++) {
        sum += i;
    }
    return sum;
}
```

第四步，用小输入检查。 `n=4` 时，四轮分别把 `0`、`1`、`2`、`3` 加进去，返回 `6`。结束时 `i==4`，这一轮只检查条件，不再加 `4`。`n=0` 时第一次条件就不成立，返回 `0`；这验证了入口跳转不能省略。这里对 `%eax`、`%edx` 写零会把整个对应 64 位寄存器的高半部分也清零，因此确实初始化了 `%rax` 与 `%rdx`。

可以再用一句话检查整个循环：每次到达 `.Lsum_test` 时，`sum` 等于 `0` 到 `i-1` 的和。初始 `i=0` 时是空和零；循环体加入当前 `i` 后再加一，下一次检查时这句话仍成立。这样的句子叫循环不变式，它比“看起来像求和”更能保证你没有漏掉最后一次迭代。

2. switch：先找入口，再沿代码走

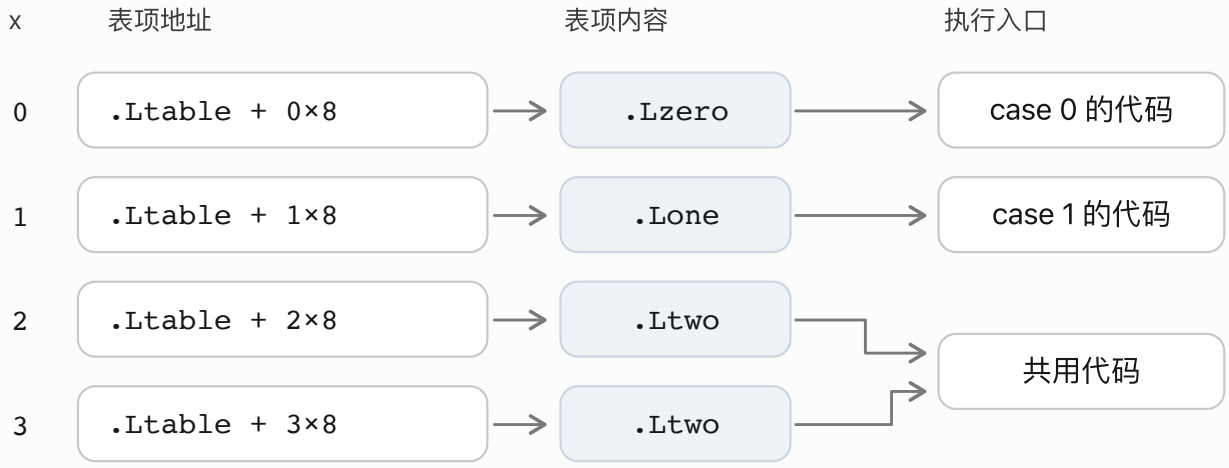
C 的 `switch` 根据一个整数值选择起点。`case` 是入口标签，`break` 才负责退出。两个相邻 `case` 可以共用同一段代码；某个 `case` 后没有 `break`，就会继续执行下一段，这称为 **fall-through，落入**。

不能看到 `case 0` 就只看它下面第一条语句。正确方法是：先用值找到入口，再沿实际控制流走到 `break`、`return` 或其他跳转为止。同理，汇编中两个不同表项可以填同一个标签；这是共用代码，不能因此认定表坏了。

2.1 跳转表里面存的是去哪里，不是计算结果

连续、比较密集的分支值适合用表定位入口。以下是本讲示例使用的绝对地址跳转表：每项用 `.quad` 存一个 8 字节代码地址。

跳转表：先读地址，再跳转



先定位表项地址，再读取其中保存的代码地址；索引 2 和 3 共用同一个入口。

读图要分三层：先计算表项的地址，再从该地址取出代码地址，最后跳到该代码地址执行。表项地址不等于表项内容；它也不是 `switch` 最终返回的数。这里的乘数 `8` 来自表项是 8 字节地址，和被操作的 C 数据是 `int` 还是 `long` 没有直接关系。

```
jmp *.Ltable(,%rdi,8)
```

这条指令用 `%rdi` 作索引，到 `.Ltable+8*%rdi` 处读一个目标地址，再跳过去。前面的 `*` 表示间接跳转；`jmp .Ltwo` 则是直接跳到给定标签。还有 `jmp *%rax`，目标地址直接来自寄存器值。不要把 `jmp *%rax` 和“去 `%rax` 所指的内存里再取一次目标”混为一谈。

先看case为0～3的表。 只有x=0、1、2、3才能直接用作这张表的索引；x<0或x>3时应去default。这里是“或”，不是两个条件同时成立。`cmpq $3,%rdi; ja .Ldefault` 按无符号解释检查x是否大于3：x=2时不跳，继续查表；x=4时跳到default；x=−1的64位位模式按无符号读为$2^{64}-1$，也跳到default。因此一次无符号上界检查就能排除范围两端。default是未匹配case时执行的代码，不代表固定返回某个数。

再换一个例子：case为10～13。 如果只放四个表项，表的索引仍为0、1、2、3，就要先算 `索引=x-10`：

原始x，即case值	索引x−10	查表去向
10	0	表[0]→case 10
11	1	表[1]→case 11
12	2	表[2]→case 12
13	3	表[3]→case 13

此时按无符号检查的是机器计算出的**索引**是否大于3，而不是原始x是否大于3。x=9得到低64位全1的索引，按无符号看很大，去default；x=14得到索引4，也去default；x=12得到索引2，进入case 12。减10只是调整查表编号，可以另用寄存器保存索引。读汇编恢复原case时要反推 `case值=索引+10`，所以表[2]对应case 12，不是case 2；这不要求程序额外执行一次加10。Q022：什么时候走default，为什么索引要减10、还原case又要加10？ 分别解释两个范围检查与原值/索引的区别。

有些机器代码用 4 字节相对偏移作表项，再加上基址得到目标，而不使用本节的 8 字节绝对地址表。遇到题目先看给定的表项格式；“跳表每项一定 8 字节”不是普遍规则。

示例 D3-E02：有落入和共享入口的 switch（往年题改编）

题源：2022 年期中无答案版（来源资料未随仓库分发），第三题，PDF 第 9—10 页。本例借用其“跳转表、落入、算术分支”的组织方式，减少分支、改小常数，去掉结构体写入，并明确所有路径的返回值。以下是改编后的完整题面与解答，不是原题原文。

参数是 `long x,long b,long c`，分别在 `%rdi,%rsi,%rdx`；取 `-10<=x<=10`、`-100<=b,c<=100`。

```
pick_value:
    cmpq $3, %rdi
    ja .Lpick_default
    jmp *.Lpick_table(,%rdi,8)
.Lpick_zero:
    subq $4, %rdx
.Lpick_one:
    leaq (%rsi,%rdx), %rax
    leaq (%rax,%rax,2), %rax
    ret
.Lpick_two:
    leaq 10(%rdx), %rax
    ret
.Lpick_default:
    movq %rsi, %rax
    ret

.section .rodata
.align 8
.Lpick_table:
    .quad .Lpick_zero
    .quad .Lpick_one
    .quad .Lpick_two
    .quad .Lpick_two
```

先列映射：0 去 `.Lpick_zero`，1 去 `.Lpick_one`，2 和 3 都去 `.Lpick_two`，范围外去默认分支。再看从每个入口出发执行了什么。

从 0 进入时先把 `c` 减 4，随后没有任何跳走指令，所以继续执行 `.Lpick_one`。两条 `lea` 先算 `b+c`，再算三倍，最后返回。从 1 进入时直接算三倍，**不会**先减 4。2 和 3 共用 `c+10`；默认返回 `b`。由此得到：

```
long pick_value(long x, long b, long c) {
    switch (x) {
    case 0:
        c -= 4;
        /* 有意落入 case 1 */
    case 1:
        return 3 * (b + c);
    case 2:
    case 3:
        return c + 10;
    default:
        return b;
    }
}
```

用 b=2,c=7 检查：x=0 返回 15，x=1 返回 27，x=2 和 x=3 都返回 17，x=-1 返回 2。共享入口只保证 2、3 执行同一段；0、1 虽然最后执行同一段，但 0 先改了 c，所以结果不同。

2.2 看见 cqto 和 idivq，不要继续把 rdx 当原参数

课程 switch 示例中有有符号除法。idivq divisor **执行前**读取一个 128 位被除数，它由 %rdx:%rax 拼成：%rdx 是高 64 位，%rax 是低 64 位。**执行后**商写入 %rax、余数写入 %rdx，覆盖它们原来的内容。高低位说的是输入，商和余数说的是输出；并非同一时刻保存两套值。冒号表示拼接位串，不是除号。

> Q023：rax、rdx为什么既是被除数的高低位，又是商和余数？用17÷5逐步看执行前后

若要把普通有符号 long a 除以 long b，通常先把 a 放 %rax，再用 cqto 把 %rax 的符号扩展到高半部分 %rdx。cqto 本身不做除法，也不改变 %rax；它会覆盖 %rdx，正数时补零，负数时补全一。

```
# 计算 a / b：a 在 %rdi，b 在 %rsi
movq %rdi, %rax
cqto
idivq %rsi
ret
```

例如 a=-13,b=5，商是 -2、余数是 -3，满足 -13=(-2)*5+(-3)。商向零截断，所以不能直接把它当算术右移的向下取整。除数不能为零；有符号最小值除以 -1 的商也无法表示。读题时遵守这些前提。

若除数原来恰好在 %rdx，必须在 cqto 前移到其他寄存器，否则会被覆盖。例如第三个参数 b 在 %rdx 时，可先 movq %rdx,%rcx，之后用 idivq %rcx。这是“寄存器的含义会随指令改变”的一个典型例子：不能从参数列表一直给 %rdx 贴着 b 的标签。

2.3 PC 相对跳转，只记一条计算规则

某些直接跳转在机器码中不保存完整目标地址，而保存一个有符号位移。计算规则是：**目标地址 = 该跳转指令结束后下一条指令的地址 + 位移**。例如跳转指令位于 0x400100，长度为 2 字节，位移为 -6，目标就是 0x400102-6=0x4000fc。

不能用当前指令的起始地址直接加位移，也不能默认所有跳转都只有两个字节；题目给了机器码长度就按长度算。负位移常见于循环回边。PC 相对跳转的“相对”说的是目标地址的编码方式，和“是否有条件”“是否是函数调用”不是同一个分类维度。

3. 调用：把“去哪里”和“带什么”分开

y=f(x) 看起来只是一句 C，机器至少要解决三件事：把控制流交给 f，把参数给 f，并保证 f 完成后能回到这句调用之后。函数若还需要局部数据，就要考虑它们放在哪里、何时不再使用。

返回地址回答“执行完后回到哪里”，它是代码地址。**返回值**回答“函数算出了什么”，例如 f(3) 得到的 9。本讲整数返回值通常在 %rax；返回地址由 call 放入栈中。返回地址既不是“函数入口地址”，也不是“调用指令本身的地址”。

调用者与被调用者是相对身份：如果 A 调 B、B 又调 C，B 对 A 是被调用者，对 C 是调用者。寄存器保存题一定要先说清正在看哪一次调用，否则会把责任颠倒。

3.1 栈是一片内存，rsp 是地址，不是数据盒子

把栈理解成按后进先出使用的一段内存。x86-64 中，栈向较低地址增长，%rsp 保存当前栈顶的地址。**低地址增长**意味着新压入一个 8 字节对象时，%rsp 减 8；不是因为数值变小就“释放了空间”。

以下描述针对常见的寄存器操作数；本讲不讨论以 %rsp 自身为 pop 目的等特殊情形。

指令	常见操作的含义
`pushq %rbx`	先把 `%rsp` 减 8，再把 `%rbx` 中保存的值复制到新栈顶的 8 字节内存；`%rbx` 不变
`popq %rbx`	从 `%rsp` 指向的内存读 8 字节，写入 `%rbx`，再把 `%rsp` 加 8
`call f`	将紧接在本条 call 后面的指令地址压栈，再开始执行函数 `f` 入口处的指令
`ret`	从当前栈顶读出返回地址，`%rsp` 加 8，再从读出的代码地址继续执行

`pushq %rbx` 保存的是寄存器里的值，不是“寄存器自身的地址”，也不会去读取 `%rbx` 所指向的内存。若 `%rbx=42`，压入的就是 42；若它保存一个指针，压入的才是那个指针值。`popq %rbx` 取回的是栈内存里的内容，而不是栈顶地址。`ret` 返回的是执行位置，普通整数返回值通常由 `%rax` 提供；若函数使用过额外栈空间，须在 `ret` 前让栈顶重新指向返回地址。

> Q024：push/pop保存的是值还是地址？call转到f、ret返回分别在做什么？

`pop`、`ret` 使栈顶越过一块内存，不代表那块内存立刻被清零；它只是退出当前活跃栈区。`subq $16,%rsp` 也只是预留 16 字节，不负责把它们初始化。考试中若内存从未写过，就不能擅自给它填零。

3.2 一次 call 的三个时刻

假设调用指令开始前 `%rsp=0x1000`，调用后的下一条指令地址记作 `R`。被调用函数先不做任何额外压栈操作。

一次 call 与 ret 的三个时刻

A. 调用前

`0x1000`
原有栈数据 ← rsp

`0x0ff8`
本次调用尚未使用

此时还没有执行 call；rsp 指向 0x1000，返回地址 R 尚未压入本次调用的栈区。

B. 刚进入被调用函数

`0x1000`
原有栈数据

`0x0ff8`
R ← rsp

call 先将 rsp 减 8，再在 0x0ff8 写入返回地址 R；控制流转到被调用函数。

C. 执行 ret 之后

`0x1000`
原有栈数据 ← rsp

`0x0ff8`
旧字节可能还在

ret 从 0x0ff8 取出 R 并使 rsp 加 8；控制回到 R。弹出不意味着旧内存被清零。

每栏中地址由高到低排列。粗边框指出 rsp 当前指向的位置；弹栈不等于清空旧字节。

按图追踪两个变化：`call` 让 `%rsp` 从 `0x1000` 变为 `0x0ff8`，且写入 `R`；`ret` 从 `0x0ff8` 取出 `R`，让 `%rsp` 恢复到 `0x1000`。被调用函数若在中间用了额外空间，必须在 `ret` 前把栈顶恢复到存放 `R` 的位置，否则 `ret` 会把别的数据误当成地址。

4. ABI：函数之间约定怎么交接

ABI 是二进制层面的约定。在本讲使用的 System V AMD64 ABI 下，普通整数或指针类参数的前六个位置如下。这里只处理简单标量参数；浮点和部分结构体有自己的分类规则，下一讲再讲。

参数次序	1	2	3	4	5	6
寄存器	`%rdi`	`%rsi`	`%rdx`	`%rcx`	`%r8`	`%r9`

第七个及后面的普通 `long` 参数通过栈传递。**刚进入被调用函数、尚未改变 `%rsp` 时**，`0(%rsp)` 是返回地址，第七个参数位于 `8(%rsp)`，第八个位于 `16(%rsp)`。如果函数已经 `push` 或 `sub` 过 `%rsp`，偏移必须相应重算，不能永远写 `8(%rsp)`。

可以用从右往左 `push` 的方式构造栈参数：先第八个，再第七个，这样第七个离返回地址最近。编译器也能一次预留空间后用 `mov` 填入，因此“参数一定由 push 指令传递”同样不成立。调用者负责回收它自己为这些栈参数准备的空间。

4.1 为什么有的值跨一次 call 就丢了

寄存器数量有限，被调用函数要用它们工作。假设A调用B，A是调用者、B是被调用者；它们使用同一组寄存器，进入B时不会自动换一套。ABI按“谁负责保留旧值”将寄存器分成两类：

- **调用者保存**：`%rax,%rcx,%rdx,%rsi,%rdi,%r8,%r9,%r10,%r11`。B可以改它们，无须恢复；如果A在B返回后还需要其中旧值，由A在调用前保存到安全位置。
- **被调用者保存**：`%rbx,%rbp,%r12,%r13,%r14,%r15`。函数可以使用它们，但若改了值，就要在返回前恢复成进入时的值。`%rsp` 也必须恢复到满足正常返回的位置，但通常不把它当一般临时寄存器列在这组里。

“调用者保存”不等于调用前必须无条件保存全部寄存器；只保存后面还需要的值。“被调用者保存”也不等于每次进入函数必须压入全部六个；没修改就不需要保存。

例如A的 `%rdi=10`，B把它改成99后可以直接返回，所以A若还需要10，要提前另存。若A的 `%rbx=10`，B想把它改成99，B就要先保存10，并在返回前恢复为10；A可以依赖这个约定。保存和恢复是程序指令完成的，不是call/ret自动备份整组寄存器。

> Q025：调用者保存、被调用者保存，究竟是谁保存、什么时候需要保存?

假设你在 `%rdi` 保存 `x`，调用 `f` 后还要用 `x`。即使 `f` 恰巧在某一次运行中没改 `%rdi`，也不能仅凭 ABI 保证它不变。稳妥做法是把 `x` 存到当前函数的栈空间，或放入自己已经妥善保存的 `%rbx`。后一种选择意味着当前函数也要对它的调用者负责，在最后恢复原 `%rbx`。

4.2 栈帧与 rbp：常见布局不是强制模板

一次尚未返回的函数调用，可以有属于它的保存信息、局部数据和临时空间。把这部分组织称作栈帧，有助于追踪递归。但编译器可能省略不需要的空间，直接用寄存器计算；也可能省略帧指针。因此不能背成“每个函数一定 `push %rbp`，每个局部变量都在栈上”。

先把栈帧理解成一次函数调用在栈上使用的区域。`%rbp`本身是寄存器；采用帧指针时，它保存这块区域里的一个固定基准地址，方便用偏移定位数据。`%rsp`保存当前栈顶地址，会随着栈空间的使用而变化。

> Q026：栈帧和rbp是什么？用具体地址逐步拆解函数开头与leave/ret

一种常见开头是：

```
pushq %rbp
movq %rsp, %rbp
subq $16, %rsp
```

执行后 `%rbp` 固定指向保存的旧 `%rbp`。在这种具体布局里，`0(%rbp)` 是旧 `%rbp`，`8(%rbp)` 是返回地址；局部数据通常在负偏移处。`%rsp` 会随着临时压栈而变，`%rbp` 则提供一个相对稳定的基准。

对应结尾可以写 `movq %rbp,%rsp; popq %rbp; ret`，其中前两条的效果可由 `leave` 完成。`leave` **不负责跳回调用者**，仍然需要 `ret`。本讲练习会明确使用哪种栈帧，不能将这张布局强套到没有帧指针的函数上。

如果代码需要把局部变量的地址交给一个实际执行内存访问的外部函数，就必须提供可寻址的内存位置，栈是常见选择。但若编译器能把整个过程内联、消除，源代码中写了 `&x` 也不保证最终机器码一定保留栈槽。考试给出汇编时，以实际指令为准；分析一个普通局部整数时，先看它是否始终只放在寄存器里。

4.3 栈对齐：看清是在 call 前还是入口处

对于本讲只用普通整数、指针的调用情形，**执行 `call` 之前，`%rsp` 应为 16 的倍数**。`call` 再压入 8 字节返回地址，所以**刚进入被调用函数时，`%rsp` 对 16 取模为 8**。两句话并不矛盾，它们描述不同时间点。更强对齐要求的特殊参数不在本讲练习范围内。

例：入口 `%rsp=0x1008`；`pushq %rbx` 后变成 `0x1000`，这时已按 16 对齐；再 `subq $16,%rsp` 后是 `0x0ff0`，仍然对齐，可以调用其他函数。如果入口直接减 16，就得到 `0x0ff8`，仍余 8，不能仅因为“分配的字节数是 16”就认为调用点对齐了。

对齐不是多画一个框就完成的。解题时记录每条 `push/pop/sub/add/call/ret` 的增减量，并在每个 `call` 前检查一次模 16。为了满足对齐而预留的空位不是自动出现的新 C 变量，也不能因为某个栈槽存在就推断它有特定内容。

示例 D3–E03：局部变量取地址，调用后再使用（助手补充）

先看两个 C 函数，输入 `-100<=x<=100`：

```c
long bump(long *p, long delta) {
    long old = *p;
    *p = old + delta;
    return old;
}

long use_bump(long x) {
    long v = 20;
    long old = bump(&v, 3);
    return x + v + old;
}
```

`bump` 返回修改前的值，同时通过指针修改 `v`。因此必须分清“返回的 `old`”和“写回内存的新值”。以下是一个符合本讲约定的实现：

```asm
bump:                       # 函数入口：rdi存指针p，rsi存增量delta
    movq (%rdi), %rax        # 读取*p的旧值到rax，留作返回值old
    addq %rsi, (%rdi)        # 将delta加到*p上，更新内存中的v
    ret                     # 返回use_bump，rax中仍是旧值old

use_bump:                   # 函数入口：rdi存输入x
    pushq %rbx              # 保存外层调用者的rbx，返回前要恢复
    subq $16, %rsp          # 预留16字节栈空间，并保持调用前16字节对齐
    movq %rdi, %rbx         # 将x存入rbx，让它在调用bump后仍可用
    movq $20, 8(%rsp)       # 在栈内存中初始化局部变量v=20
    leaq 8(%rsp), %rdi      # 将v的地址作为第1个参数p，不是读取v的值
    movl $3, %esi           # 设置第2个参数delta=3；写esi也将rsi高32位清零
    call bump              # 调用bump(&v,3)；返回后rax=20，内存中v=23
    addq 8(%rsp), %rax      # 将修改后的v加到old上：rax=20+23=43
    addq %rbx, %rax         # 再加输入x，得到返回值x+v+old=x+43
    addq $16, %rsp          # 回收预留的16字节，让rsp指向保存的旧rbx
    popq %rbx               # 恢复外层的rbx，并让rsp指向外层返回地址
    ret                     # 返回外层调用者，rax中保留计算结果x+43
```

先问每个位置装什么。 `%rbx` 保存输入 `x`，供调用后使用；`8(%rsp)` 是 `v` 的内存位置；`lea` 把这个位置的地址给第一个参数 `%rdi`，没有读取 `v`；第二个参数 `%rsi` 是 3。若错写 `movq 8(%rsp),%rdi`，传过去的是数值 20，被当指针使用就完全不是原程序了。

再追一次栈。 假设进入 `use_bump` 时 `%rsp=0x1008`，此时 `0x1008` 已放着它返回到外层的地址 `R外`。



图中地址由高到低排列。`R内` 是 `call bump` 后第一条 `addq` 的地址。`bump` 入口的 `%rsp` 改变了，但 `%rdi` 内保存的地址仍是 `0x0ff8`，不会自动跟着 `%rsp` 变。它将该处从 20 改成 23，并把旧值 20 放在 `%rax`。

`bump` 的 `ret` 弹出 `R内`，返回 `use_bump` 后 `%rsp` 又是 `0x0ff0`。两条加法得到 `20+23+x`，所以 `x=5` 时返回 48。随后释放 16 字节、恢复原 `%rbx`，`%rsp` 回到 `0x1008`，最后 `ret` 才弹出 `R外`。

这里没有使用 `%rbp`，照样能管理栈。保存 `%rbx` 是为了对外层调用者履行义务；把 `x` 放进去是为了让 `bump` 按约定保住它。两个理由对应不同层次，必须同时说清。

5. 递归：重复的是代码，分开保存的是每次调用的状态

递归就是函数调用自己。没有另一套特殊 `recursive call` 指令，仍使用 `call` 和 `ret`。难点在于同一段代码同时有多次调用尚未结束，每次都需要知道“我自己的参数是什么、子调用返回后接着做什么”。

先把递归拆成两阶段。**向下调用**时，当前层暂时停住，保存稍后要用的值，进入下一层；**向上返回**时，子层把结果放到 `%rax`，当前层取回自己保存的值，完成剩下的计算，再返回。函数名一样不代表局部状态共用一个槽。

示例 D3-E04：每一层都要保留自己的 n（助手补充）

```c
/* 0 <= n <= 100 */
long sum_to(long n) {
    if (n == 0) return 0;
    return n + sum_to(n - 1);
}
```

一种实现如下：

```asm
sum_to:
    testq %rdi, %rdi
    je .Lsum_base
    pushq %rbx
    movq %rdi, %rbx
    subq $1, %rdi
    call sum_to
    addq %rbx, %rax
    popq %rbx
    ret
.Lsum_base:
    movl $0, %eax
    ret
```

入口先检查 `n==0`。基本情形直接返回零，没有额外保存 `%rbx`；因此基本情形路径也不能执行 `popq %rbx`，否则会把返回地址错当保存值弹掉。非基本情形先保存进入本层时的旧 `%rbx`，再把本层 `n` 放到 `%rbx`；把参数改为 `n-1` 后递归调用。

为什么不直接等调用回来再加 `%rdi`？它是调用者保存寄存器，而且我们已经把它减一传给子层，不能再用它当本层原来的 `n`。为什么子层也使用 `%rbx` 却不会破坏本层？因为子层同样先保存进入时的 `%rbx`，返回前恢复。这条约定层层成立，便形成各层独立状态。

对 `sum_to(3)`，把调用和返回分开写：



读图时先从左上向右下走，直到基本情形；再从右下往上代入结果。最大同时活跃调用数为 4，调用自己形成的递归边数为 3。这两个数不同。虽然基本情形没有额外 `push %rbx`，它依然有由调用它的 `call` 压入的返回地址。

每个非基本层入口的 `%rsp` 对 16 余 8，执行一次 `push %rbx` 后余 0，恰好满足再调用自己的对齐要求。返回时 `pop` 与本层的 `push` 配对。这里不能随意把 `push` 删掉再说“只用一个寄存器就够了”，它同时参与保存旧值与调用点对齐。

5.1 双递归还多一个问题：第一个结果也要保住

考虑 `f(n)=f(n-1)+f(n-2)`。第一次 `call` 的结果在 `%rax`；第二次 `call` 也会把自己的结果放进 `%rax`。如果不保存第一个结果，最后只有第二个结果了。因此，两次调用之间必须把第一次结果转移到栈槽或妥善使用的被调用者保存寄存器。

这种问题可见于 2023 年期中带答案版（来源资料未随仓库分发）第三题，PDF 第 16—17 页的组合数递归代码。本讲只指出其训练价值，不要求今天把整道长题全部补完。遇到多次 `call`，给每个跨越 `call` 仍然活跃的值画一条“必须保存”的标记，比盲目给每行写中文更有效。

还要区分“总调用次数”与“最大栈深”。两个递归调用通常先后完成，不是同时在栈上铺开一整棵树。最大栈深看一条尚未返回的调用链；总调用次数才统计整棵调用树。今天先练单链递归，确保每层状态和返回顺序能独立写对。

6. 遇到长题时，按这个顺序落笔

循环题先给变化寄存器命名，找到初始化与回边，再翻译比较的方向，最后用零、一次循环、恰好到边界三个输入核对。若反推的 C 和原作者写法不同，只要行为和题目要求一致，不必为了猜原代码而把正确解改错。

switch 题先记参数寄存器，再看索引是否平移和是否检查范围，逐项列出“索引→标签”，最后从标签沿代码执行到终点。先完成映射表再算结果，能避免漏掉共享入口和落入。出现 `idiv` 时，单独标出 `%rax/%rdx` 被重新赋予的含义。

调用题先画当前 `%rsp` 和返回地址，再逐条记栈顶变化；参数和返回值另写在寄存器旁边。对于每次 `call`，只问三件事：参数现在对不对？调用后还需要的值保存了吗？此刻栈是否对齐？`ret` 前再核对栈顶是否指向正确返回地址。

递归题先做最小输入。若基本情形还不清楚，直接追大输入只会把错误放大。然后画两层，明确每层保存什么、子结果放在哪里，最后才推广到更多层。不要用“递归会自动保存所有寄存器”作解释；只有给定指令和 ABI 约定承担这些工作。

7. 独立练习：4 道必做，共 40 分

以下均为助手补充的新题。先只看题面，写出推理；每题 10 分，四题合计 40 分，基础过关线为 32 分。除总分外，必须至少独立画对一次调用前、入口、返回后的栈变化。得到提示后完成的部分可以订正，但不计入首次独立正确率。每题的分值在小问后给出，评分过程另见答案文件。

D3-P01：恢复循环并核对边界（10 分）

`long loop3(long n)` 的输入满足 `0<=n<=100`，`n` 在 `%rdi`：

```
loop3:
    movl $0, %eax
    movl $1, %edx
    jmp .Lp1_test
.Lp1_body:
    leaq (%rdx,%rdx,2), %rcx
    addq %rcx, %rax
    addq $2, %rdx
.Lp1_test:
    cmpq %rdi, %rdx
    jle .Lp1_body
    ret
```

1. 说明 `%rax` 和 `%rdx` 分别承担什么角色。（1 分）
2. 写出初始化、继续循环的条件、每轮更新。（2 分）
3. 写出等价且完整的 C 函数，不能直接用闭式求和公式替代循环。（2 分）
4. 对 `n=5`，列出每轮加入的数及最终返回值。（1 分）
5. 对 `n=0`，是否执行循环体？说明依据。（1 分）
6. 只把 `jle` 改成 `jl`，`n=5` 时返回值如何变化？（1 分）
7. 写出一句在每次到达 `.Lp1_test` 时成立的关系，说明当前累计值与 `%rdx` 的关系，并解释为什么更新后仍成立。（2 分）

D3-P02：跳转表、默认分支与落入（10 分）

参数 `long x,long a` 在 `%rdi,%rsi`，输入 `-10<=x<=10`、`-100<=a<=100`。给定：

```
route:
    cmpq $4, %rdi
    ja .Lp2_default
    jmp *.Lp2_table(,%rdi,8)
.Lp2_a:
    addq $2, %rsi
.Lp2_b:
    leaq (%rsi,%rsi,4), %rax
    ret
.Lp2_c:
    leaq -3(%rsi), %rax
    ret
.Lp2_default:
    movq $-1, %rax
    ret

.section .rodata
.align 8
.Lp2_table:
    .quad .Lp2_a
    .quad .Lp2_b
    .quad .Lp2_default
    .quad .Lp2_c
    .quad .Lp2_c
```

1. 写出 x=0,1,2,3,4 分别进入的标签。(2 分)
2. 为何 x=-1 也会去默认分支？说明 ja 的比较方式。(1 分)
3. 哪些 x 共用入口？哪一处存在落入？ (1 分)
4. 写出完整等价 C 函数，显式表示 case 与默认分支。(2 分)
5. 分别计算 route(0,4)、route(1,4)、route(3,4)。(1 分)
6. 假设表基址是 0x5000，x=4 时读哪个表项地址？该处存的是结果还是代码地址？ (1 分)
7. 若将间接跳转换成 leaq .Lp2_table(,%rdi,8),%rax; jmp *%rax，还能正确执行吗？说明错在“取表项地址”和“读表项内容”的哪一步。(2 分)

D3-P03：局部内存、调用前后与栈恢复（10 分）

以下 caller 输入 -100<=x<=100，x 在 %rdi。进入 caller 时 %rsp=0x2008，内存 [0x2008] 为返回到外层的地址 R0，原 %rbx=99。把 call read_add 后的第一条指令地址记为 R1。未明确写入的栈内存内容未知。

```
caller:
    pushq %rbx
    subq $16, %rsp
    movq %rdi, %rbx
    movq $7, 8(%rsp)
    leaq 8(%rsp), %rdi
    movl $5, %esi
    call read_add
    addq %rbx, %rax
    addq $16, %rsp
    popq %rbx
    ret

read_add:
    movq (%rdi), %rax
    addq %rsi, %rax
    ret
```

1. 执行 pushq %rbx、subq $16,%rsp 后，分别给出 %rsp；保存的 99 在哪个地址？（2 分）
2. 紧挨 call 之前，%rdi 和 %rsi 的值分别是什么？该处是否满足 16 字节对齐？（1 分）
3. 刚进入 read_add 时，%rsp 是多少？R1 在哪个地址？画出 R0、旧 %rbx、局部值 7 与 R1 的相对位置。（2 分）
4. 对 x=4，read_add 返回值与 caller 最终返回值分别是多少？（1 分）
5. read_add 返回到 caller 后，及 caller 最终返回外层后，%rsp 分别是多少？最终 %rbx 为多少？（2 分）
6. 如果删去保存 x 到 %rbx 的做法，并将调用后的加法改成 addq %rdi,%rax，能否得到 x+12？指出本段具体代码里 %rdi 已经变成什么，并说明一般情况下的 ABI 风险。(2 分)

D3-P04：递归每层保存什么（10 分）

odd_sum(long n) 的输入满足 0<=n<=20。给定：

```
odd_sum:
    testq %rdi, %rdi
    je .Lp4_base
    pushq %rbx
    leaq -1(%rdi,%rdi), %rbx
    subq $1, %rdi
    call odd_sum
    addq %rbx, %rax
    popq %rbx
    ret
.Lp4_base:
    movl $0, %eax
    ret
```

1. 写出基本情形与递推关系。(2 分)
2. 写出完整等价 C 函数。(1 分)
3. 计算 odd_sum(3)，分别列出向下各非基本层保留的值和向上各层返回的值。(2 分)
4. 为什么递归调用后还能用 %rbx？“被调用者保存”在父层和子层各起什么作用？（1 分）
5. 计算 odd_sum(3) 时，最多同时活跃几个本函数调用？其中有几个非基本层的 push %rbx 尚未匹配 pop？（1 分）
6. 假设进入最外层 odd_sum(3) 时 %rsp=0x3008，到 odd_sum(0) 刚入口时 %rsp 是多少？说明每下降一层的变化，并检查非基本层递归调用前的对齐。(2 分)
7. 基本情形路径为什么不能照搬非基本路径执行一次 popq %rbx 再 ret？（1 分）

D3-P05：超过六个参数的调用（选做，10 分）

已完成四道必做题并能正确画栈后再做。本题不计入前面的 40 分。假设紧挨第一条 push 前 %rsp=0x4000，前六个 long 参数 1—6 已正确放入六个参数寄存器：

```
    pushq $8
    pushq $7
    call eight
    addq $16, %rsp
    # 回到调用者的后续代码

eight:
    movq 8(%rsp), %rax
    addq 16(%rsp), %rax
    addq %rdi, %rax
    ret
```

1. 列出前六个参数的寄存器。(1 分)
2. 两次压栈后、刚进入 `eight` 时的 `%rsp` 分别是多少？ (2 分)
3. 在 `eight` 入口画出返回地址、第七、第八参数的位置。(2 分)
4. `eight` 返回什么值？它修改了哪些寄存器？是否破坏任何被调用者保存寄存器？ (2 分)
5. 为什么需要调用后的 `addq $16,%rsp`？执行后栈顶是多少？ (1 分)
6. 如果交换两次 `push` 的顺序，参数布局会怎样变化？为什么仅凭这次返回值未改变，也不能说交换是正确的？ (2 分)

8. 本讲完成时应留下什么

留下四道题的首次答案、至少一张调用栈图，以及实际出现的错误。若循环条件错了，记录具体是哪条 `cmp` 被读反；若栈错了，记录是哪一次 8 字节变化遗漏；若递归错了，记录哪个跨调用的值没有保存。不要只写“汇编不熟”或“递归不会”，这样的标签无法指导第二次练习。

分数达到 32 分后，再检查三个必须能口述的结论：为什么跳表要先读表项再跳转；返回地址和返回值分别在哪里；为什么父子递归层都使用 `%rbx` 仍能各自保留状态。总分够而这三点不清楚时，先回到对应示例重画一次，不追加更长递归。

来源与范围核对

- 课程第 5 讲：Control（来源资料未随仓库分发）：PDF 第 37—47 页为循环，第 49—58 页为 switch、跳转表及除法，第 62—64 页为反汇编中的跳转表。
- 课程第 6 讲：Procedures（来源资料未随仓库分发）：PDF 第 10—24 页为栈和控制传递，第 26—27 页为参数，第 29—53 页为栈帧与局部存储，第 54—65 页为保存约定，第 67—75 页为递归及总结。
- 第 5 讲普通研讨题（来源资料未随仓库分发）：研讨 2(3)(5)—(9)；第 6 讲普通研讨题（来源资料未随仓库分发）：研讨 1(4)(5)(7)、研讨 2(1)—(4)(6)—(8)。这里只借用课内概念要求，不安排外部调研或拓展任务。
- 教材对应 3.4.4（压栈与弹栈）、3.5.5（特殊算术操作）、3.6.3—3.6.4（跳转指令与编码）、3.6.7—3.6.8（循环和 switch）、3.7（过程）。教材 PDF（来源资料未随仓库分发）仅按小节定位，不猜填印刷页码。

除标明的往年题改编外，本讲示例、练习、图示与讲解均为助手补充；涉及的机器规则以所述课程环境为前提。往年题带答案材料只作核对来源，不作为未经检查的正确性保证。